

Osc-Mamba: Chaotic Neural Oscillator-Based Mamba for Sequential Recommendation

Zhang Xinyu¹, Xiao Zheng¹, Zhang Minghao¹, and Raymond S. T. Lee^{1,†}

¹Guangdong Provincial Key Laboratory IRADS, Faculty of Science and Technology,
Beijing Normal-Hong Kong Baptist University, Zhuhai 519000, Guangdong, China
s230034061@mail.bnbu.edu.cn, s230032047@mail.bnbu.edu.cn,
s230026211@mail.bnbu.edu.cn

[†]Corresponding author: raymondshtlee@bnb.edu.cn

Abstract. Sequential recommendation aims to model the dynamic preferences of users from historical behavior sequences for accurate item prediction. State space models (SSM) such as Mamba have demonstrated strong capability in long-sequence processing with linear complexity. However, in sequential recommendation, Mamba exhibits limited sensitivity to high frequency and transient local dynamics, suffers from insufficient nonlinear expressivity, and fails to capture the oscillatory nature present in complex user preference evolution. To address this limitation, we introduce Osc-Mamba, an effective framework that enhances Mamba for recommendation with a chaotic oscillatory mechanism (Lee-oscillator). The oscillator introduces frequency-sensitive nonlinear responses that adaptively regulate hidden-state transitions, improving the model’s feature representation ability. We design three strategies to integrate oscillatory dynamics with Mamba. Experimental results demonstrate that our approach outperforms competitive baselines in three sequential recommendation datasets.

Keywords: Sequential Recommendation · State Space Models · Lee Oscillator · Dynamic Activation · Mamba

1 Introduction

With the rapid expansion of digital information, recommendation systems have become essential tools for alleviating information overload and improving user experience [16, 12]. Sequential recommendation focuses on modeling the temporal order of user interactions to capture evolving preferences, enabling accurate next-item prediction in domains such as e-commerce, social media, and entertainment [6, 23].

Early sequential models based on Markov chains and collaborative filtering [9] struggled to capture long-range dependencies. Subsequent neural models based on RNNs, CNNs, and Transformers—such as GRU4Rec [7], SAS-Rec [8], and BERT4Rec [19] significantly advanced sequence modeling but suffer from quadratic computational cost. Recently, structured State Space Models (SSM) have emerged as efficient backbones for long-range temporal modeling [15]. Mamba [3] and its recommendation variant Mamba4Rec [13] achieve

linear-time computation while maintaining strong representation ability. However, SSM-based sequential recommendation models typically employ static activation functions (e.g., ReLU, GELU), which cannot adjust their nonlinear behavior to reflect abrupt interest shifts, noise, or local fluctuation patterns in user behaviors [18]. Real-world preference trajectories often exhibit oscillatory, non-stationary, or chaotic dynamics [1, 21], which static activations cannot adaptively capture.

Although SSM effectively encodes long-range dependencies, they do not explicitly capture short-term oscillations or unstable preference transitions. For example, a user consistently viewing action thrillers may temporarily engage with a documentary and then immediately return to their initial high frequency genre. Such intermittent and wave-like patterns resemble dynamic systems with excitatory-inhibitory interactions [22]. Inspired by biologically chaotic neural oscillator, the Lee Oscillator [10, 11] provides adaptive nonlinear transformation through iterative excitatory inhibitor loops and energy dependent gating. These mechanisms have shown effectiveness in modeling non-stationary temporal signals [25, 17].

To address the limitations of Mamba in sequential recommendation and motivated by the chaotic neural oscillator, we propose **Osc-Mamba**, a new architecture that integrates the Lee Oscillator with Mamba. This integration introduces dynamic, input-dependent nonlinear activation while preserving the efficiency and long-range modeling capability of the underlying state-space backbone. We investigate three oscillator integration strategies: **1)** Applying the oscillator after item embedding. **2)** Appending Lee-oscillator after each Mamba block. **3)** Using the Lee Oscillator as the activation function in the Feedforward network. The key contributions of Osc-Mamba model are to:

1. We propose **Osc-Mamba**, the first model in sequential recommendation to integrate chaotic neural oscillators with selective state-space models, enhancing both efficiency and dynamic nonlinear expressiveness.
2. We develop three oscillator integration strategies and conduct a systematic analysis to explore the best place and mechanism of the oscillator.
3. Experiments on ML-1M, Amazon-Beauty, and Amazon-Video-Games demonstrate that our model achieves overall superior performance compared to Mamba4Rec and other baselines in HR@10, NDCG@10, and MRR@10, validating the benefit of oscillatory dynamics in sequential recommendation.

2 Related Work

Sequential Recommendation Early sequential models relied on Markov chains and pattern mining [7], but struggled with long-term dependencies. With the development of deep learning, RNN-based architectures such as GRU4Rec [6] and its successors were used to capture temporal dynamics through recurrent hidden states. The method of CNN was later adapted [20] to identify salient interaction patterns using convolutional filters. The advent of Transformers revolutionized

the field. This architecture represented a significant leap forward, with SASRec [8] establishing the standard self-attention mechanism for sequential recommendation, and BERT4Rec [19] introducing bidirectional context modeling via cloze-task training. To handle computational challenges in long sequences, recent research has focused on improving efficiency and capturing specific temporal aspects: LinRec [14] proposed a linear attention variant for long sequences, while FEARec [2] incorporated frequency-domain analysis to model global periodic patterns.

SSM and Mamba State Space Models (SSM) provide an alternative to attention by modeling sequences through continuous-time latent state transitions. Structured State Space Models [4] demonstrated that SSM with specific initialization schemes could achieve remarkable performance on long-range reasoning tasks. Compared with attention mechanisms, SSM also offers superior computational efficiency. Mamba [3] introduces selective SSM whose parameters dynamically adapt to input tokens, achieving linear-time sequence modeling with strong expressiveness. Its efficiency has attracted interest in recommendation settings, where long user histories are common. Mamba4Rec [13] demonstrates how Mamba can be integrated with positional encoding and feedforward components to build a scalable sequential recommendation architecture.

Chaotic Neural Oscillator Research on chaotic neural oscillators aims to simulate dynamic phenomena in the biological brain. Early works by Aihara et al. [1] pioneered chaotic neural networks for associative memory. The recurrent unit EINS [22] combines excitatory and inhibitory interactions with chaotic behavior through modeling biological neural synaptic activity. Lee [10] proposed an innovative chaotic neural oscillator (Lee-oscillator), which realizes a progressive memory recall mechanism and gradually reconstructs the memory pattern through the chaotic associative process. The CT2TFDNN [11] further validates that these oscillatory mechanisms can effectively capture multi-scale temporal patterns in complex time series, providing the theoretical foundation for our model. Previous works have shown the effectiveness of oscillator in popular deep learning models. Examples include the ReLU-OSC [18] for chaotic activations in CNN and CNO-FSGNN [25] for GNN-based node classification. CANOE [24] demonstrated the practicality of chaotic neural oscillators for next-location prediction. In this work, we innovatively incorporate the Lee Oscillator into the Mamba state-space architecture. By introducing the excitatory-inhibitory mechanism to modulate Mamba’s internal updates, Osc-Mamba shows dynamic, input-dependent, non-linearity that significantly enhances the model’s ability to capture periodic fluctuations and rapid temporal patterns in user interaction sequences.

3 Methodology

3.1 Framework Overview

The overall architecture of Osc-Mamba is shown in Figure 1. Given an input item sequence, we obtain item embeddings, process them through a Mamba block, and

apply a Feedforward network. The final prediction layer outputs ranking scores for next-item recommendation. Our main design applies the Oscillator at the embedding level (The Lee Oscillator is applied to item embeddings before the Mamba block), allowing dynamic nonlinear modulation to propagate through the entire Mamba architecture. The two variants are: (1) Mamba-level: The Oscillator is inserted after the Mamba block, and its output is sent to the Feedforward network. (2) FFN-level: The Oscillator is used as the activation function inside the Feedforward network.

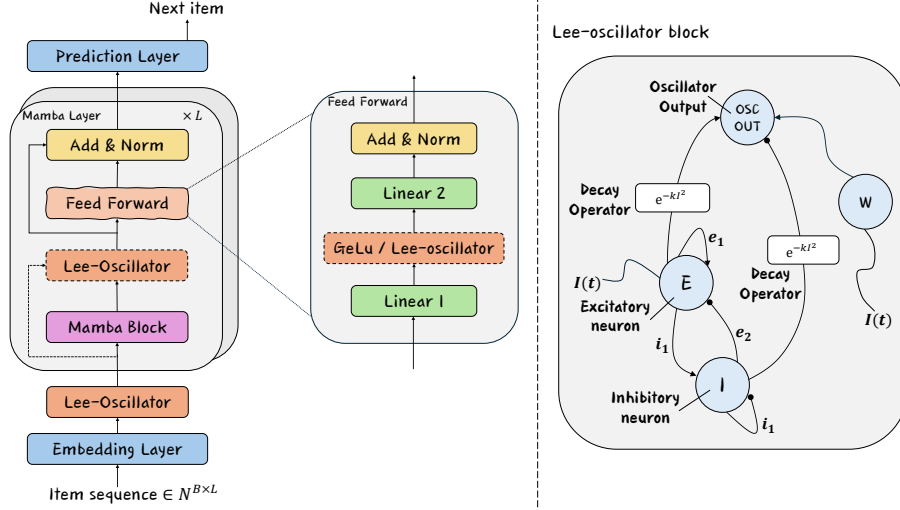


Fig. 1: Overall Framework of Osc-Mamba and the architecture of Lee-oscillator block used in our model

3.2 Embedding Layer

Embedding layers are essential components in sequential recommendation systems, as they project sparse user-item interaction sequences into a rich, high-dimensional latent space. In our proposed architecture, user interaction sequences are similarly mapped into embeddings, serving as the critical first stage for feature representation. Given a user interaction sequence $S_u = [x_1, x_2, \dots, x_T]$ of item IDs, the embedding layer maps each item to a D -dimensional embedding using a learnable matrix $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times D}$. The embedded sequence is:

$$\mathbf{X} = [\mathbf{e}_{x_1}, \mathbf{e}_{x_2}, \dots, \mathbf{e}_{x_T}] \in \mathbb{R}^{T \times D}. \quad (1)$$

To improve robustness, embedding dropout and layer normalization are applied:

$$\mathbf{H} = \text{LayerNorm}(\text{Dropout}(\mathbf{X})) \in \mathbb{R}^{T \times D}. \quad (2)$$

During training, sequences from multiple users are padded to a maximum length L , resulting in input $\mathbf{H} \in \mathbb{R}^{B \times L \times D}$ for a mini-batch of size B .

3.3 Mamba Layer

A Mamba layer combines a selective state-space Mamba block and a position-wise feed-forward network:

$$\mathbf{H}' = \text{MambaBlock}(\mathbf{H}), \quad (3)$$

$$\mathbf{H}_1 = \text{LayerNorm}(\mathbf{H} + \text{Dropout}(\mathbf{H}')), \quad (4)$$

$$\mathbf{H}_2 = \text{FFN}(\mathbf{H}_1). \quad (5)$$

Assuming that the Lee-oscillator is not used after the embedding layer, the output of the embedding layer is directly passed to the Mamba block. The Mamba block is based on structured state-space dynamics with convolutional projection and selective input gating. It uses SiLU activation and linear projections to efficiently encode temporal dependencies. The FFN is a two-layer MLP with hidden dimension $4D$ and GELU activation in the original Mamba4Rec[13] implementation. For a single-layer setting, the residual connection around the Mamba block can be removed, while for multiple layers residual connections are retained to ease optimization.

3.4 Prediction layer

For each user sequence, the final hidden state at the last non-padded position is taken as the sequence representation:

$$\mathbf{h}_u = \mathbf{H}_L[T^*] \in \mathbb{R}^D \quad (6)$$

where T^* is the index of the last valid interaction. $\mathbf{H}_L$ is the output of the final Mamba layer stack. The score for candidate item i is obtained by the inner product between $\mathbf{h}_u$ and the item embedding $\mathbf{e}_i$. Training uses either Bayesian Personalized Ranking (BPR) loss or cross-entropy loss with sampled negatives; in our implementation we follow the cross-entropy setting.

3.5 Chaotic Neural oscillator

To boost the model’s nonlinear representational capabilities and facilitate adaptive feature modulation, we introduce the Lee Oscillator, which is specifically designed to address the limitations of static activation functions in capturing complex time series dynamics[10]. Given an input tensor $\mathbf{I} \in \mathbb{R}^{B \times L \times D}$, the module maintains two internal states: an excitatory variable $\mathbf{u}$ and an inhibitory variable $\mathbf{v}$ with the same shape as $\mathbf{I}$. The right graph in Figure 1 shows the architecture of Lee-oscillator.

Iterative excitatory–inhibitory updates. The oscillator performs K iterations of coupled updates:

$$\mathbf{u}^{(m+1)} = \text{ReLU} \left(a_1 \mathbf{u}^{(m)} + a_2 \mathbf{v}^{(m)} + \mathbf{I} \right), \quad (7)$$

$$\mathbf{v}^{(m+1)} = \text{ReLU} \left(b_1 \mathbf{u}^{(m+1)} - b_2 \mathbf{v}^{(m)} \right), \quad (8)$$

where a_1, a_2, b_1, b_2 are learnable scalar parameters and $m = 0, \dots, K - 1$. In the implementation, $\mathbf{u}^{(0)}$ and $\mathbf{v}^{(0)}$ are initialized as zero tensors, and k is a learnable scalar that determines the extent to which the model relies on the input signal’s energy ($\mathbf{I}_2$) to scale the amplitude of the oscillatory component. This grants the oscillator dynamic nonlinear characteristics. These recurrent updates allow the module to build richer nonlinear patterns than a single static activation.

Energy-controlled exponential gate. After the iterative updates, the oscillator computes an energy term based on the input:

$$\mathbf{I}_2 = \text{mean}_d(\mathbf{I} \odot \mathbf{I}) \in \mathbb{R}^{B \times L \times 1}, \quad (9)$$

where the mean is taken over the feature dimension d . A learnable scale k controls an exponential gate:

$$\mathbf{g} = \exp(\text{clamp}(k \cdot \mathbf{I}_2, -30, 30)). \quad (10)$$

The final oscillatory output is then given by

$$\mathbf{Z} = \tanh(\mathbf{u}^{(K)} - \mathbf{v}^{(K)}) \odot \mathbf{g} + \text{ReLU}(\mathbf{I}). \quad (11)$$

A residual connection and layer normalization stabilize the transformation:

$$\hat{\mathbf{Z}} = \text{LayerNorm}(\mathbf{I} + \text{Dropout}(\mathbf{Z})). \quad (12)$$

This design mirrors biological oscillatory circuits: the excitatory–inhibitory loop shapes the phase and amplitude, while the energy gate controls how strongly the nonlinearity is expressed for each position in the sequence.

3.6 Proposed Osc-Mamba Model

We integrate the Lee Oscillator with the Mamba architecture through three strategies and we conduct comprehensive experiments to evaluate and analyze all three integration strategies (In the Experiment section). The introduction of the Lee Oscillator provides an inductive bias that enhances the input-dependent gating behaviors of the Mamba Block, enabling it to respond more effectively to subtle local sequential variations.

Chaotic Neural oscillator after embedding Firstly, we explore placing the oscillator after the embedding layer:

$$\mathbf{H}_0 = \text{LayerNorm}(\text{Dropout}(\mathbf{X})), \quad (13)$$

$$\mathbf{H}_1 = \text{Osc}(\mathbf{H}_0), \quad (14)$$

$$\mathbf{H}_2 = \text{MambaLayer}(\mathbf{H}_1). \quad (15)$$

This pre-modulation strategy treats the oscillator as a learnable gate that amplifies informative interactions and suppresses noise before any state-space propagation. In practice, we find this variant particularly useful on datasets with high item diversity and noisy sequential dynamics such as MovieLens-1M and Amazon-Video-Games.

Chaotic Neural oscillator after Mamba block In this variant, each Mamba block processes the sequence following the original backbone design, after which the Lee-oscillator is applied to the Mamba block output:

$$\mathbf{H}_1 = \text{MambaBlock}(\mathbf{H}), \quad (16)$$

$$\mathbf{H}_2 = \text{Osc}(\mathbf{H}_1). \quad (17)$$

This allows the oscillator to reshape the long-range temporal patterns produced by the SSM before passing them to the next layer or the prediction head.

Chaotic Neural oscillator as activation function In this variant, we keep the linear layers of the original feed-forward network but replace its GELU activation with the oscillator:

$$\text{FFN}_{\text{osc}}(\mathbf{H}) = \text{LayerNorm}(\mathbf{H} + \text{Dropout}(\mathbf{W}_2 \text{Dropout}(\text{Osc}(\mathbf{W}_1 \mathbf{H})))) \quad (18)$$

where W_1 and W_2 are linear projections and $\text{Osc}(\cdot)$ denotes the Lee Oscillator applied along the feature dimension. This directly substitutes the static point-wise nonlinearity with an adaptive oscillatory one.

All three variants share the same prediction layer and the training objective is the same as the baseline Mamba4Rec[13]. During the experiments, we compare their behavior under a unified experimental setup.

4 Experiment

4.1 Experiment Setup

Datasets We evaluate Osc-Mamba on three benchmark datasets commonly used in sequential recommendation:

- **MovieLens-1M (ML-1M)**[5] A movie rating dataset containing approximately one million interactions.
- **Amazon-Beauty**¹ A subset of the Amazon product review corpus focusing on beauty products.
- **Amazon-Video-Games**² Another Amazon subset containing interactions with video game items.

Following the preprocessing settings of Mamba4Rec and Recbole[26], we treat ratings as implicit feedback and build user interaction sequences ordered by timestamp. For all datasets, we adopt the same filtering strategy as the released configuration: users and items with fewer than 5 interactions are removed, and sequences are truncated to a maximum length specified by `MAX_ITEM_LIST_LENGTH` (200 for ML-1M and 50 for the Amazon datasets). For each user, we follow the

¹ https://cseweb.ucsd.edu/~jmcauley/datasets.html\#amazon_reviews

² https://cseweb.ucsd.edu/~jmcauley/datasets.html\#amazon_reviews

leave-one-out protocol implemented in RecBole[26]: the last item is used for testing, the second last for validation, and the remaining interactions for training.

Baselines We compare Osc-Mamba against a set of widely used sequential recommendation models, including GRU4Rec[6], BERT4Rec[19], SASRec[8], LinRec[14], FEARec[2], and the Mamba4Rec[13] baseline. These models cover recurrent-based, Transformer-based, linear sequence modeling, feature-enhanced architectures, and selective state-space methods, allowing us to comprehensively evaluate the effectiveness of our oscillator-based modifications.

Evaluation Metrics We evaluate model performance using three widely adopted ranking metrics in sequential recommendation: Hit Rate@10 (HR@10), Normalized Discounted Cumulative Gain@10 (NDCG@10), and Mean Reciprocal Rank@10 (MRR@10). HR@10 measures whether the ground-truth item appears within the top-10 positions. The metrics are defined as follows:

- **HR@10** measures whether the ground-truth item appears within the top-10 positions.
- **NDCG@10** rewards models that rank the correct item higher by applying a logarithmic discount to lower positions.
- **MRR@10** evaluates the reciprocal rank of the first relevant item, emphasizing early correct predictions.

All metrics are computed with a ranking cutoff of 10.

Implementation Details We use a single NVIDIA RTX 4090 GPU for all the experiments. The method of data processing and program architecture are based on the RecBole framework [26]. For optimization, we choose the Adam optimizer with learning rate of 0.0001. And the embedding dimension we set for all the baselines and our model is 64. Following are the implementation details for the Lee-oscillator block: The gating coefficient k is initialized to 10.0. For the MovieLens-1M and Amazon-Video-Games datasets, the iteration number is 4. For the Amazon-Beauty datasets, the iteration number is 5. All the other hyperparameters inside this block are default numbers of Lee-oscillator [10]. Other training settings and evaluation setting are the same as the Mamba4Rec[13] to ensure fairness.

Method	MovieLens-1M			Amazon-Beauty			Amazon-Video-Games		
	HR@10	NDCG@10	MRR@10	HR@10	NDCG@10	MRR@10	HR@10	NDCG@10	MRR@10
GRU4Rec	0.2931	0.1645	0.1247	0.0597	0.0329	0.0248	0.1025	0.0538	0.0378
BERT4Rec	0.2958	0.1673	0.1289	0.0693	0.0394	0.0281	0.1053	0.0536	0.0382
SASRec	0.2980	0.1684	0.1296	0.0762	0.0421	0.0315	0.1126	0.0570	0.0391
LinRec	0.3042	0.1710	0.1318	0.0745	0.0413	0.0314	0.1139	0.0583	0.0395
FEARec	<u>0.3165</u>	0.1731	<u>0.1426</u>	0.0807	0.0425	0.0319	<u>0.1142</u>	0.0591	0.0418
Mamba4Rec	0.3137	<u>0.1824</u>	0.1422	<u>0.0764</u>	<u>0.0448</u>	<u>0.0353</u>	0.1127	<u>0.0602</u>	<u>0.0443</u>
Osc-Mamba	0.3233*	0.1856*	0.1436*	0.0753	0.0459*	0.0371*	0.1168*	0.0629*	0.0462*

Table 1: **performance comparison.** Best results are **bold**, second-best are underlined, and * denotes statistically significant improvements of Osc-Mamba over the Mamba4Rec baseline ($p < 0.01$).

4.2 Overall Performance

As shown in Table 1, we compare our proposed Osc-Mamba with several competitive baselines in three datasets. In general, Osc-Mamba achieves outstanding performance with significant improvements ranging from 0.98% to 5.10%. The model establishes state-of-the-art results on the MovieLens-1M and Amazon-Video-Games datasets and achieves highly competitive performance on the Amazon-Beauty dataset. Specifically, on the large-scale MovieLens-1M dataset, Osc-Mamba significantly outperforms all other baselines, including the Mamba4Rec baseline across all three metrics. The most notable gains are observed on the Amazon-Video-Games dataset, where Osc-Mamba surpasses the second best method by considerable margins (2.29% relative improvement in HR@10, 4.49% in NDCG@10, and 4.29% in MRR@10 over Mamba4Rec). These improvements demonstrate the effectiveness of the Lee-oscillator in Mamba architecture. While the FEARec achieves the highest HR@10 score on the Amazon-Beauty datasets, showing its strength in area. The optimal integration strategy is different between the three datasets. For MovieLens-1M and Amazon-Video-Games, placing the oscillator module after the embedding layer yielded the best results. For Amazon-Beauty, the most effective architecture is to use the Lee-oscillator as the activation function in the Feedforward network. This indicates that Lee-oscillator could strengthen early-stage sequential signals for dense and highly structured datasets and also enhance the latent preference signals learned by the model for highly sparse, long-tail datasets. More analysis of different mechanisms is provided in the ablation study section.

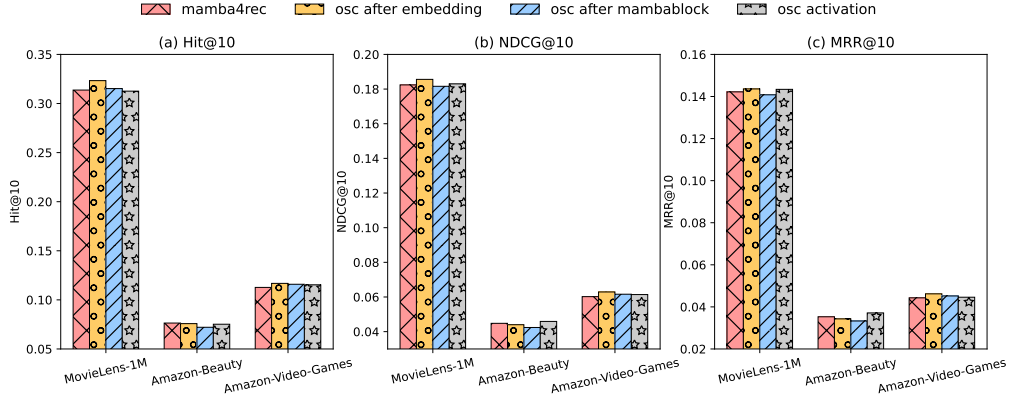


Fig. 2: Ablation study results across the three architectures.

4.3 Ablation Study

In this section, we conducted ablation experiments across the three datasets to evaluate the impact of three distinct Lee-oscillator integration strategies. Figure ?? compares the performance of these architectures: **1.** Lee-oscillator after the embedding block. **2.** Lee-oscillator after the Mamba block. **3.** Lee-oscillator

as the activation function in the feedforward network block. Details are illustrated in the Methodology section.

It is evident that for both MovieLens-1M and Amazon-Video-Games, Oscillator After Embedding architecture demonstrates the best performance, showing that the oscillator effectively performed dynamic modulation of the input features and enables the SSM to capture more complex, dynamic, and long-range dependencies. In terms of the Amazon-Beauty datasets, the oscillator activation mechanism outperforms baselines in NDCG@10 and MRR@10 most significantly, maintaining comparable performance in Hit@10. The possible reason is that the user sequences are more structured for movies and games datasets so the oscillator after the embedding layer could strengthen early-stage sequential signals. In contrast, user behaviors in Amazon-Beauty are more concentrated and discrete, exhibiting static and dependent rather than dynamic and periodic patterns. While the oscillator activation mechanism could improve the model’s non-linear expressive power, thereby enhancing its ability to accurately rank the next item. Placing the oscillator after the Mamba block yields moderate improvements on Amazon-Video-Games, but provides limited benefits on the other two datasets. Table (b) in Table 2 shows the results of the ablation study on Amazon-Video-Games. Apart from the first best effect, both **I₃** (osc after Mamba block) and **I₃** (osc activation) also bring noticeable improvements over the baseline, with **I₂** achieving gains of 2.03% to 2.84% and **I₃** bringing improvements of 0.68% to 2.31% across all metrics.

4.4 Model Efficiency

We analyze the GPU memory utilized during model training and the training time consumed. The comparison results of Model efficiency on the Amazon-Beauty dataset between Osc-Mamba and other baselines are shown in table (a) in Table 2. Our proposed Osc-Mamba model maintains the inherent efficiency advantages of the Mamba architecture. Specifically, Mamba-based models use nearly an order of magnitude less GPU memory than the self-attention model BERT4Rec, confirming the superior memory efficiency of the Selective State Space Model. Osc-Mamba is slightly slower than Mamba4Rec due to the additional computation from the oscillator, but it still maintains a much faster speed than BERT4Rec and also faster than FEARec. This demonstrates that Osc-Mamba strikes an effective balance between performance and efficiency, achieving superior results with minimal resource increase.

5 Conclusion and Future work

In this paper, we propose Osc-Mamba, which enhances Mamba for sequential recommendation by introducing the Lee-oscillator to better capture oscillatory patterns inherent in user interaction sequences. To examine how the placement of the Lee-oscillator influences performance, we design three architectural variants: applying the oscillator after the embedding layer, after the Mamba block, and

(a) Model efficiency comparison on Amazon-Beauty. (b) Ablation study on Amazon-Video-Games.

Model	Train Time	GPU Mem.	Comp.	HR@10	NDCG@10	MRR@10
GRU4Rec	61s	1.99GB	w/o osc	0.1127	0.0602	0.0443
BERT4Rec	812s	12.58GB	I₁	0.1168	0.0629	0.0462
FEARec	223s	2.01GB	I₂	0.1159	0.0616	0.0452
Mamba4Rec	208s	1.08GB	I₃	0.1153	0.0614	0.0446
Osc-Mamba	254s	1.29GB				

Table 2: Comparison of model efficiency and ablation study results. **I₁**, **I₂**, **I₃** stand for osc after embedding, osc after Mamba block and osc activation respectively.

using it as an activation function. We conduct comprehensive ablation studies and evaluate both model effectiveness and efficiency. Experiments on MovieLens-1M, Amazon-Beauty, and Amazon-Video-Games demonstrate notable improvements over Mamba4Rec and other strong baselines in HR@10, NDCG@10, and MRR@10, while preserving linear-time efficiency. We also observe that the optimal integration strategy varies across datasets with different characteristics. In future work, we will further explore the expressive oscillators for better capturing sequential representations and adapt our model to broader range of tasks.

References

1. Aihara, K., Takabe, T., Toyoda, M.: Chaotic neural networks. *Physics letters A* **144**(6-7), 333–340 (1990)
2. Du, X., Yuan, H., Zhao, P., Qu, J., Zhuang, F., Liu, G., Liu, Y., Sheng, V.S.: Frequency enhanced hybrid attention network for sequential recommendation. In: *Proceedings of the 46th international ACM SIGIR conference on research and development in information retrieval*. pp. 78–88 (2023)
3. Gu, A., Dao, T.: Mamba: Linear-time sequence modeling with selective state spaces. In: *First conference on language modeling* (2024)
4. Gu, A., Goel, K., Ré, C.: Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396* (2021)
5. Harper, F.M., Konstan, J.A.: The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)* **5**(4), 1–19 (2015)
6. Hidasi, B., Karatzoglou, A., Baltrunas, L., Tikk, D.: Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939* (2015)
7. Hidasi, B., Tikk, D.: General factorization framework for context-aware recommendations. *Data Mining and Knowledge Discovery* **30**(2), 342–371 (2016)
8. Kang, W.C., McAuley, J.: Self-attentive sequential recommendation. In: *2018 IEEE international conference on data mining (ICDM)*. pp. 197–206. IEEE (2018)
9. Koren, Y., Bell, R., Volinsky, C.: Matrix factorization techniques for recommender systems. *Computer* **42**(8), 30–37 (2009)
10. Lee, R.S.: Lee-associator—a chaotic auto-associative network for progressive memory recalling. *Neural Networks* **19**(5), 644–666 (2006)

11. Lee, R.S.: Chaotic type-2 transient-fuzzy deep neuro-oscillatory network (ct2tfdnn) for worldwide financial prediction. *IEEE Transactions on Fuzzy Systems* **28**(4), 731–745 (2019)
12. Linden, G., Smith, B., York, J.: Amazon. com recommendations: Item-to-item collaborative filtering. *IEEE Internet computing* **7**(1), 76–80 (2003)
13. Liu, C., Lin, J., Wang, J., Liu, H., Caverlee, J.: Mamba4rec: Towards efficient sequential recommendation with selective state space models. *arXiv preprint arXiv:2403.03900* (2024)
14. Liu, L., Cai, L., Zhang, C., Zhao, X., Gao, J., Wang, W., Lv, Y., Fan, W., Wang, Y., He, M., et al.: Linrec: Linear attention mechanism for long-term sequential recommender systems. In: *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. pp. 289–299 (2023)
15. Orvieto, A., Smith, S.L., Gu, A., Fernando, A., Gulcehre, C., Pascanu, R., De, S.: Resurrecting recurrent neural networks for long sequences. In: *International Conference on Machine Learning*. pp. 26670–26698. PMLR (2023)
16. Resnick, P., Varian, H.R.: Recommender systems. *Communications of the ACM* **40**(3), 56–58 (1997)
17. Shi, N., Chen, Z., Chen, L., Lee, R.S.: Cno-lstm: A chaotic neural oscillatory long short-term memory model for text classification. *IEEE Access* **10**, 129564–129579 (2022)
18. Shi, N., Chen, Z., Chen, L., Lee, R.S.: Relu-oscillator: Chaotic vgg10 model for real-time neural style transfer on painting authentication. *Expert Systems with Applications* **255**, 124510 (2024)
19. Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., Jiang, P.: Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In: *Proceedings of the 28th ACM international conference on information and knowledge management*. pp. 1441–1450 (2019)
20. Tang, J., Wang, K.: Personalized top-n sequential recommendation via convolutional sequence embedding. In: *Proceedings of the eleventh ACM international conference on web search and data mining*. pp. 565–573 (2018)
21. Wang, L., Pichler, E.E., Ross, J.: Oscillations and chaos in neural networks: an exactly solvable model. *Proceedings of the National Academy of Sciences* **87**(23), 9467–9471 (1990)
22. Wang, L., Lee, R.S.: Excitatory and inhibitory neuronal synapse unit: A novel recurrent cell for time series prediction. *Neurocomputing* **594**, 127858 (2024)
23. Wu, S., Tang, Y., Zhu, Y., Wang, L., Xie, X., Tan, T.: Session-based recommendation with graph neural networks. In: *Proceedings of the AAAI conference on artificial intelligence*. vol. 33, pp. 346–353 (2019)
24. Wu, Y., Peng, Y., Yu, J., Liu, X., Yan, Z., Lin, K., Su, W., Qu, B., Lee, R., Yang, D.: Beyond regularity: Modeling chaotic mobility patterns for next location prediction. *arXiv preprint arXiv:2509.11713* (2025)
25. Zhang, L., Lee, R.S.: Chaotic neural oscillators with deep graph neural network for node classification. In: *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. pp. 168–180. Springer (2024)
26. Zhao, W.X., Mu, S., Hou, Y., Lin, Z., Chen, Y., Pan, X., Li, K., Lu, Y., Wang, H., Tian, C., et al.: Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. In: *proceedings of the 30th acm international conference on information & knowledge management*. pp. 4653–4664 (2021)